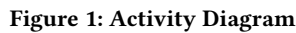


Sofia Constantinou, Dimitar Zhekov, Nicole Almeida, Stefanos Kritikos, Alex Christou, Efe Koç
Eindhoven University of Technology, The Netherlands
2127326, 2132966, 2087480, 2153785, 2075407, 2098156, Group 58

1.1 Activity Diagram



1.2 Testing Strategy

Requirement	Test Case	Passing Criteria
1.4.1.(1) The system must display the levels, distinguishing between the ones that are unlocked and locked.	Unit tests should individually test the LevelDTO constructor and setters to verify proper ordering, default locked state for Level 1. Unit tests should isolate and test the unlocking logic in the progression manager. Integration tests verify overall unlocking sequence and manual unlock behavior.	System must provide 50 sequentially numbered LevelDTOs (1-50). Unit tests should confirm Level 1 always shows unlocked and unlocking transitions N to N+1 correctly. Other levels reflect actual progression state. Manual unlocks grant immediate access. Level 50 remains locked until progression.

Requirement	Test Case	Passing Criteria
1.4.1.(2) The system must provide the user with the ability to start playing a level once selected.	A unit test should isolate level loading logic: verify LevelLoader class loads the correct level ID, assets, and unlock state separately. Unit tests should simulate level selection and readiness for interaction. Manual testing confirms level interactions respond within 3 seconds.	Unit tests should confirm LevelLoader preserves level number and unlocked state, Selected level should load all assets and accept player input within 3 seconds of selection.
1.4.1.(3,4) The system must have background music and in-game sounds. The system must provide the user with the ability to adjust the volume.	Unit tests should verify sound playback with valid and invalid paths, concurrent clip playback, and edge cases such as empty and null paths. Unit tests should isolate AudioManager methods for setVolume, and getVolume separately. Unit tests should also verify setting and retrieving master volume with persistence checks, automatic clamping of out-of-bounds values, and independent control of music/SFX channels.	System must ensure all sounds play within 100ms of triggering, ensure invalid paths fail gracefully, ensure background music loops seamlessly, volume setter/getter unit tests validate precision to ± 0.001 and clamping between 0.0-1.0, settings persist after level restart, and independent music/SFX channels do not interfere.
1.4.1.(5) The system must provide the user with the ability to choose their avatar out of a predefined set.	Unit tests verify the ability to set and retrieve the player's avatar selection using selectedAvatar method and verify persistence of the avatar chosen.	Selected avatar should appear consistently across all game views and persist after restart. Unit tests confirm correct Avatar ID is stored, retrieved, and persisted. Avatar changes should render in all applicable views within 1 second of selection.
1.4.1.(6) The system must have a settings button, which leads to a view with the options to adjust the sound volume, choose an avatar, exit the settings and exit to the levels menu.	Unit tests verify volume controls, avatar updates, settings persistence, and independent audio channels. They also isolate and test navigation logic. Manual tests confirm all settings paths are accessible and responsive.	System applies volume changes instantly ± 0.001 precision, retains avatar after reload, saves settings within 1 second, and auto-corrects invalid values. Unit tests confirm correct navigation targets. Manual tests ensure no UI lag and correct view transitions.